LedgerLens: Agentic Invoice Exception Investigation

Ayaan Khan | MiraFlores Case Study | Prepared for GreenGrowth CPAs

Draft status: The full real-model workflow has passed validation. Public hosting and the final prototype link are pending. This document is not the final submission.

Source and setup: https://github.com/ayaan-27/Miraflores_Project/tree/feat/agentic-invoice-investigator

1. The problem I chose and what motivated it

I considered three problems related to work I have done.

Connecting a decision to its supporting evidence. At Ricoh, I worked on a document-retrieval assistant that preserved references between service instructions and supporting diagrams. That experience motivated my interest in workflows where retrieving an answer is only the beginning: someone must inspect its evidence and decide what to do. In accounting, I focused on invoice exceptions that require a reviewer to compare documents and records, identify what is missing, and follow up. If that work is fragmented, reviewers can repeat checks and clients or suppliers can receive unclear requests.

Getting reliable answers across databases. I have worked on a text-to-SQL system at Ricoh spanning more than 50 relational databases and a curated knowledge graph. That work motivated my interest in how schema differences and business definitions affect an answer. Operational users can depend on specialists to retrieve information or interpret results. I considered a narrowly scoped analytics assistant to reduce that dependence.

Preparing useful data without exposing sensitive information. At Mphasis, I worked on synthetic-data generation and sensitive-information redaction. Those projects motivated my interest in safer evaluation datasets. Model builders need representative data, while data owners need to control disclosure. Restricted access can slow evaluation; uncontrolled sharing creates privacy risks.

I chose invoice exception investigation because it allowed a small but complete workflow: read an invoice, investigate a discrepancy, explain the evidence, and produce a review decision or request. An analytics product would need validated database semantics, while a data-sharing product would need a substantial privacy-and-utility evaluation. Here I could test a concrete handoff using synthetic documents and records.

I have not interviewed GreenGrowth's staff or observed its internal process. My previous work informs the design; the frequency and cost of this accounting problem remain assumptions.

2. How I thought about the workflow and the role of agents

My starting question was: what work happens after an exception is flagged? A reviewer needs to know why an invoice is blocked, which evidence supports that conclusion, and what would resolve it. I narrowed the prototype to an invoice whose billed quantity exceeds the available receiving evidence.

I broke the workflow into four stages: read and ground the invoice; investigate with evidence tools; challenge the findings; pause for human review. New evidence can return the same case to investigation. This makes the result a documented resolution path rather than an isolated warning.

My working assumption is that a reviewer currently moves among accounting records, documents, a spreadsheet, and email. Existing products may already solve much of this. A better outcome would be less repeated checking, clearer requests, and a traceable decision history. I would compare the prototype with that actual process before building integrations.

Why LangGraph. The case needs explicit shared state, conditional tool execution, feedback when evidence is incomplete, and a pause that survives a process restart. LangGraph provides those execution and checkpoint primitives. The local model chooses read-only investigation tools based on observed results. Required checks and bounded retries constrain the workflow. A new document resumes the same case rather than starting an unrelated conversation.

Where the model helps. It extracts structured fields from PDF text, chooses evidence queries, and drafts a source-cited review narrative. Conventional code verifies source quotes, performs exact monetary calculations, checks source IDs, and controls which decisions are allowed. A reviewer cannot approve through the API while a required financial control fails.

The alternative. A standardized spreadsheet and request checklist could be cheaper to adopt. A conventional pipeline could also handle a completely fixed matching task. The graph is useful here because investigation can return for missing checks, pause for a person, accept a document, and rerun. If those loops do not reduce effort in practice, I would use the simpler workflow or add the capability to an existing platform.

The broader opportunity is a review workspace connected to real accounting systems. The first step deliberately uses one narrow invoice workflow and synthetic system records so that its behavior can be examined without production credentials.

3. What the prototype implements

Stage 1: Read and ground. The application reads text from an actual invoice PDF. A local Qwen2.5-1.5B model produces typed fields and source quotes. A source lookup can anchor a value to a unique matching labeled PDF line; it never changes the extracted value or guesses among ambiguous matches. Each value must occur in its quoted source page. Unsupported extraction is returned with specific validation feedback for up to three attempts; an unresolved error stops the run. Scanned images require OCR and are outside this version.

Stage 2: Investigate. The model chooses among tools for the purchase order, receiving records, vendor master, payment history, and policy. Tools execute real read-only queries against the synthetic fixture catalog. Each result returns to the planner. Inputs, outputs, task descriptions, and source IDs appear in the execution trace. The planner has an eight-decision limit per round.

Stage 3: Challenge. Missing mandatory tool checks return to investigation. Independent code checks invoice arithmetic, approved order details, billed quantities and prices, receiving support, bank-account suffix consistency, and possible prior payment. Monetary calculations use decimal

arithmetic. A verifier drafts a workpaper; a conflicting recommendation or nonexistent source ID triggers revision. The computed controls remain authoritative.

Stage 4: Human review and resume. The graph saves a SQLite checkpoint before review. A reviewer can record a hold, request evidence, or recommend release when all controls pass. A receiving-report reply is extracted and validated before it can change the evidence. An invalid reply preserves the prior case. A valid reply resumes investigation and retains the earlier reviewer history.

Representative case. An invoice bills $10,000 for 100 lamps. The approved order agrees, but the initial receiving report supports 80. The computed result is a hold with $2,000 lacking receiving support. The reviewer requests the missing evidence and supplies an additional report for 20 lamps. The graph reruns the investigation; the computed unsupported amount becomes zero. A new reviewer decision is still required. This is not a partial-payment instruction or proof of document authenticity.

Difficult cases and output. Other synthetic cases cover changed bank details, a possible duplicate invoice, and a clean invoice. A reviewer note cannot clear a failed bank or duplicate control. Outputs include a readable PDF workpaper, an editable evidence-request draft, and a JSON record with source metadata, tool results, control findings, and decisions. No email, payment, vendor update, or accounting entry is initiated.

The Python API, local inference, graph, tools, controls, checkpointing, and browser interface are implemented. The companies, PDFs, internal records, and policies are synthetic. OCR, actual accounting integrations, verified user identities, immutable audit storage, foreign currencies, taxes and discounts, and split or combined invoices are deferred. Bank verification compares suffixes only.

4. Validation, learning, and disclosure

Validation so far. Fifteen framework, control, checkpoint, and API tests pass using explicit model test doubles. They execute the actual graph and tools, including the hold-to-evidence-to-review loop, unsupported extraction repair, rejected citations, attempted control overrides, invalid replies, and recovery after reopening SQLite. Browser acceptance checks cover source inspection, execution trace, PDF reply, reload/history, reviewer decisions, and JSON/PDF downloads. These tests establish workflow behavior, not local-model quality.

The first genuine model test downloaded and checksum-verified the official weights and executed inference. It stopped on an unsupported SKU quote. That failure led to the bounded extraction-repair loop; it is not counted as a passing model test. The repaired workflow subsequently passed real-model integration on the pinned CPU runtime: the initial hold, ten tool calls across two rounds, the receiving-report reply, and both review gates completed in 525.7 seconds. Reviewer actions in that test were automated inputs. Public deployment must still be confirmed before this draft is finalized.

What I would learn next. I would interview three bookkeeping or accounts-payable reviewers and inspect consented, redacted examples of their current process. I want to establish whether the bottleneck is finding records, interpreting exceptions, requesting evidence, or waiting for

replies. I would compare a standardized spreadsheet with LedgerLens using five to ten willing client datasets.

I would measure reviewer minutes per 100 invoices, extraction errors, incorrect evidence assignments, request completeness, and repeated requests. A provisional goal is at least 25% less reviewer time without increasing incorrect recommendations. That is a target to test, not an achieved saving. Small synthetic examples cannot establish real-world accuracy.

When I would stop or change direction. I would narrow or stop the product if existing tools already solve the problem, preparing the inputs costs more effort than the workflow saves, model errors require excessive checking, or the spreadsheet performs equally well. If demand is established, the next steps would be reviewed document extraction, authenticated identities, durable storage, and real integrations. Free-host filesystem behavior is not a guarantee of persistent audit history.

AI, reuse, and time. I used an AI coding assistant to help develop the scope, implementation, tests, and written draft. No employer code, client data, or prior application code was reused. The inference runtime uses local open weights and requires no model API key. Runtime boundaries and test status are stated explicitly in the setup guide. The initial prototype took about 25 minutes of AI-assisted work; the agent upgrade and validation followed. Final elapsed time will be recorded after validation, excluding my personal review and video recording.